# Green University of Bangladesh
# Department of Computer Science and Engineering (CSE)
**Faculty of Sciences and Engineering**
**Semester: (Fall, Year:2024), B.Sc. in CSE (Day)**

**Lab Report NO : 01**
**Course Title: Algorithm Lab**
**Course Code: CSE 206 Section: 223-D5**

**Lab Experiment Name:** Implement BFS using an Adjacency Matrix and find the path between two nodes.
Implement DFS using an Adjacency List and check if there is a cycle in the graph.

## Student Details

| | Name | ID |
|---|---|---|
| **1.** | Md. Reahoon Zannah | 223002038 |

**Lab Date : 16/09/2024**
**Submission Date : 23/09/2024**
**Course Teacher's Name : Md. Sabbir Hosen Mamun**

## Lab Report Status

**Marks: ……** **Signature:.....**
**Comments:.....** **Date:.....**

## 1. TITLE OF THE LAB REPORT EXPERIMENT

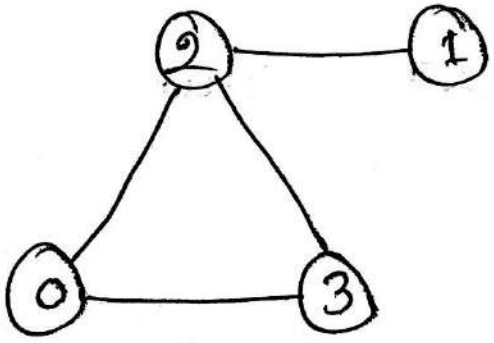
- Implement BFS using an Adjacency Matrix and find the path between two nodes.
- Implement DFS using an Adjacency List and check if there is a cycle in the graph.
- The input for the graph should be sourced from a file.

## 2. OBJECTIVES/AIM

- **BFS (Breadth-First Search) with an Adjacency Matrix**: This implementation aims to find the shortest path between two nodes in a graph by utilizing the adjacency matrix representation, which provides an efficient way to determine connections between nodes.

- **DFS (Depth-First Search) with an Adjacency List**: This part focuses on detecting cycles in the graph using an adjacency list, which is memory-efficient for sparse graphs, and allows us to explore graph nodes deeply to identify back edges indicating cycles.

## 3. GRAPH



**4. IMPLEMENTATION**

BFS algorithm in Java

```
import java.util.*;

public class Graph {
  private int V;
  private LinkedList<Integer> adj[];
Graph(int v) {
    V = v;
    adj = new LinkedList[v];
    for (int i = 0; i < v; ++i)
      adj[i] = new LinkedList();
  }
  void addEdge(int v, int w) {
    adj[v].add(w);
  }
  void BFS(int s) {
    boolean visited[] = new boolean[V];
    LinkedList<Integer> queue = new LinkedList();
    visited[s] = true;
    queue.add(s);

    while (queue.size() != 0) {
      s = queue.poll();
      System.out.print(s + " ");

      Iterator<Integer> i = adj[s].listIterator();
      while (i.hasNext()) {
        int n = i.next();
        if (!visited[n]) {
          visited[n] = true;
          queue.add(n);
        }
      }
    }
  }

  public static void main(String args[]) {
    Graph g = new Graph(4);

    g.addEdge(0, 1);
```

```
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(2, 0);
    g.addEdge(2, 3);
    g.addEdge(3, 3);

    System.out.println("Following is Breadth First Traversal " + "(starting from vertex
2)");

    g.BFS(2);
  }
}
```

DFS algorithm in Java

```
import java.util.*;

public class DFSGraphExample {
   private int vertices;
   private List<Integer>[] adjacencyList;

   public DFSGraphExample(int vertices) {
      this.vertices = vertices;
      adjacencyList = new LinkedList[vertices];
      for (int i = 0; i < vertices; i++) {
         adjacencyList[i] = new LinkedList<>();
      }
   }

   public void addEdge(int src, int dest) {
      adjacencyList[src].add(dest);
      adjacencyList[dest].add(src);
   }

   public boolean hasCycle() {
      boolean[] visited = new boolean[vertices];
      for (int i = 0; i < vertices; i++) {
         if (!visited[i]) {
```

```
            if (dfsCycleCheck(i, visited, -1)) {
                return true;
            }
        }
    }
    return false;
}

private boolean dfsCycleCheck(int current, boolean[] visited, int parent) {
    visited[current] = true;

    for (int neighbor : adjacencyList[current]) {
        if (!visited[neighbor]) {
            if (dfsCycleCheck(neighbor, visited, current)) {
                return true;
            }
        } else if (neighbor != parent) {
            return true;
        }
    }
    return false;
}

public static void main(String[] args) {
    int vertices = 4;
    DFSGraphExample graph = new DFSGraphExample(vertices);
    graph.addEdge(0, 2);
    graph.addEdge(0, 3);
    graph.addEdge(2, 3);
    graph.addEdge(2, 1);

    System.out.println("Checking for a cycle in the graph using DFS:");
    if (graph.hasCycle()) {
        System.out.println("The graph contains a cycle.");
    } else {
        System.out.println("The graph does not contain a cycle.");
    }
}
}
```

## 5. TEST RESULT / OUTPUT

BFS output

```
Output
Note: /tmp/tyLoevTK05/Graph.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
java -cp /tmp/tyLoevTK05/Graph
Following is Breadth First Traversal (starting from vertex 2)
2 0 3 1
=== Code Execution Successful ===
```

DFS output

```
Output
Note: /tmp/nM2snBFye8/DFSGraphExample.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
java -cp /tmp/nM2snBFye8/DFSGraphExample
Checking for a cycle in the graph using DFS:
The graph contains a cycle.

=== Code Execution Successful ===
```

## 6. SUMMARY:

The task involves using BFS with an adjacency matrix to find the shortest path between two nodes in a graph and using DFS with an adjacency list to detect cycles in the graph. This demonstrates the use of BFS for pathfinding and DFS for cycle detection, utilizing different graph representations effectively.